Reward Shaping and Opponent Curricula for Sparse-Reward Multi-Agent Play in SoccerTwos

Chuye Zhang
Georgia Institute of Technology
czhang883@gatech.edu

Jianuo Qiu
Georgia Institute of Technology
jqiu317@gatech.edu

Abstract: We study PPO in SoccerTwos, where the sparse goal reward makes early learning very slow. Adding a small delta-based shaping signal was enough to get the agent to start learning: against a frozen random opponent, win rate crossed 0.9 within 0.25M environment steps, and the resulting policy won 8 of 10 matches against the CEIA baseline. We then added an opponent curriculum to push the agent further. Our submitted agent, LUCKETS_AGENT, was trained for 35.8M steps across three resumed segments, and beat the shaping-only agent 7 of 10 matches head-to-head. We also report a curriculum attempt that failed: the win rate climbed to 0.99 but the curriculum never left Stage 0, because the opponent's weights were not actually swapped in the Ray workers. Diagnosing this is what led to the final design.

Code: https://github.com/ZhangChuye/soccer-twos-starter-18
Submitted agent (single agent, covers all rubric criteria): `LUCKETS_AGENT` — Policy performance, Reward modification, Curriculum learning.

Keywords: Multi-agent RL, reward shaping, curriculum learning, PPO, SoccerTwos

1 Overview

SoccerTwos is hard for two reasons. First, only goals score, so reward is very sparse: the agent can take millions of steps before it sees a single non-zero return, and most PPO updates carry almost no useful signal. Second, self-play has no fixed training target. If both teams converge on a trivial idle strategy, neither side has any reason to break the deadlock.

We test two ideas in response. Reward shaping addresses the sparsity problem by adding a small dense signal that fires every step. An opponent curriculum addresses the non-stationarity problem by gradually shifting the training distribution, from a random opponent toward the course-provided CEIA baseline, and finally toward frozen snapshots of the agent itself [1]. We ran four training configurations to isolate the contribution of each idea: a sparse self-play baseline, reward shaping against a random opponent, a curriculum design that we initially got wrong, and the redesigned curriculum that produced the submitted agent.

2 Method

Algorithm and library. All runs use Ray RLlib [2] with the PPO trainer [3] (Ray 1.4.0, PyTorch backend). The policy and value networks share a two-layer $[256, 256]$ ReLU MLP trunk. Full hyperparameters are listed in Appendix A.

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

Reward modification. The default environment returns ±1 only at goals, plus a small per-step penalty. We wrap it with `RewardShaperWrapper` (`utils.py`), which adds two small dense rewards on every step. The first rewards the agent for getting closer to the ball. The second rewards the team whenever the ball moves toward the opponent's goal.

We use changes in distance and ball position rather than the absolute values. This matters: an absolute distance reward penalises an agent for resting away from the ball, even when standing back is the right thing to do, and the policy ends up chasing the ball instead of positioning. The delta form only rewards the agent when it actually approaches the ball, and only rewards the team when the ball actually moves forward. Concretely:

$$r_t^{\text{prox}}(i) = \alpha_{\text{prox}}\big(d_{t-1}(i) - d_t(i)\big), \qquad r_t^{\text{prog}}(i) = \alpha_{\text{prog}}\big(x_t^{\text{ball}} - x_{t-1}^{\text{ball}}\big)\,\text{sign}(\text{team}(i)) \tag{1}$$

where $d_t(i) = \|p_t^{\text{player}}(i) - p_t^{\text{ball}}\|_2$ and x^{ball} is the ball's forward coordinate. We set α_{prox}=0.02 and α_{prog}=0.05, so progress dominates proximity: pushing the ball forward is always preferred to chasing it. Both terms are nearly potential-based [4], so they bias the agent toward useful behaviour without changing the optimal policy in expectation.

Opponent curriculum. The curriculum changes who the agent plays against during training, in four stages. Stage 0 uses two random-weight opponents (`opp_rand`), so the agent first learns the basic mechanics of approaching and shooting the ball. Stage 1 mixes one random opponent with one CEIA baseline player (`opp_base`), so the policy starts seeing structured defense without facing it on both sides at once. Stage 2 replaces both opponents with the CEIA baseline. Stage 3 plays the agent against frozen snapshots of itself (`opp_self`), where the opponent grows together with the policy. Promotion is gated on the training win rate: the agent advances when it wins at least 80% of episodes for three consecutive iterations.

We see the curriculum as a way of slowly extending the training distribution. A policy trained only against random opponents converges to a narrow set of states; large parts of the CEIA opponent's behaviour are off-distribution for it. Each stage shifts the training distribution toward stronger play without making the learning problem too hard at any one step. The current stage is written to a `.curriculum_stage` file that every Ray worker re-reads on each rollout, so the orange-team policy assignment stays consistent across processes.

3 Experimental Results

We ran four training configurations. Together they trace the path that produced LUCK-ETS_AGENT. Figure 1 shows the corresponding training curves.

Phase 1: sparse self-play. Our first run was a sanity check. We trained PPO with three rotating frozen-snapshot opponents and the original sparse reward (run `67194`, 2.4M steps). The agent never scored a goal in any of the 10 evaluation matches, against either the example player or the CEIA baseline. This was the first sign that sparse reward, not the self-play structure, was the main thing to fix.

Phase 2: reward shaping vs. a random opponent. We then added the delta-based shaping wrapper and froze the orange team to a random-weight opponent (run `a6c6d`, 10M steps). Win rate crossed 0.9 by about 0.25M steps and stayed near 1.0 for the rest of training. In evaluation, this agent won 10/10 against the example player and 8/10 against the CEIA baseline. The contrast with Phase 1 was the clearest evidence we had that reward shaping—not anything about the self-play setup—was the dominant factor.

Phase 3: a curriculum that did not work. We then wrote a curriculum scheduler around a single shared `opponent` policy that we intended to remap between random, baseline, and self-play (run `7b8e4`, 15M steps). At first the run looked correct: win rate climbed to 0.99 and the promotion rule fired repeatedly, but the curriculum stage indicator never moved past 0. The cause was that replacing a shared policy's weights at runtime requires updating every Ray worker's local config, while our callback only mutated the driver's copy. The opponent

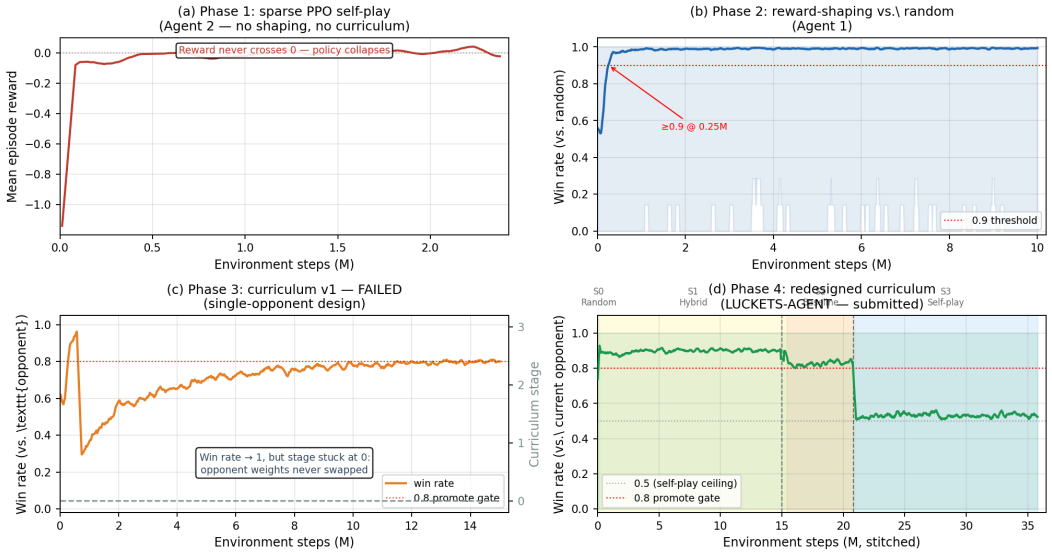

Figure 1: Training curves for the four main experiments. (a) Sparse self-play fails to score: episode reward never crosses 0. (b) Reward shaping against a random opponent quickly converges to a near-perfect win rate. (c) The first curriculum attempt reaches a high win rate, but the stage indicator (right axis) never leaves 0 – this is the policy-mapping bug. (d) The redesigned curriculum advances through all four stages across three resumed training segments (vertical dashed lines mark resumes; coloured backgrounds mark stages; red dotted is the 0.8 promotion gate; grey dotted is the 0.5 self-play ceiling). Run IDs: (a) `67194`; (b) `a6c6d`; (c) `7b8e4`; (d) `c33b8` + `6eb9a` + `02fdb`.

never actually changed, so the agent was simply learning to dominate a random-init policy more thoroughly.

Phase 4: the redesigned curriculum (LUCKETS_AGENT). We rebuilt the curriculum to declare three separate opponent policies up front (`opp_rand`, `opp_base`, `opp_self`) and select between them in `policy_mapping_fn` based on the stage file, so weight swaps stay consistent across workers. Training was split into three resumed segments because of cluster-time limits, with the stage state and checkpoint carrying across each resume: segment 1 (`c33b8`, 0−15M) covers Stages 0 and 1; segment 2 (`6eb9a`, 15−20.8M) advances through Stage 2; segment 3 (`02fdb`, 20.8−35.8M) trains at Stage 3 against frozen learner snapshots. We submit `checkpoint-600` of segment 3 as LUCKETS_AGENT.

Head-to-head evaluation. We evaluated each agent in 10 headless matches per pairing, using deterministic policies (`explore=False`) and a 1,000-step timeout. Table 1 shows the records from the blue team's perspective. LUCKETS_AGENT won 10/10 against the example player and 8/10 against the CEIA baseline; the two losses against CEIA were both shutouts. Against the shaping-only Agent 1, LUCKETS won 7, lost 1, and drew 2. Agent 1 also won 8/10 against CEIA, but with 2 draws and no losses. The pattern is consistent: LUCKETS plays more aggressively, which wins more head-to-head matches but occasionally costs a goal against a strong defender, while Agent 1 is more conservative and tends to settle for draws. Agent 2 (sparse self-play) drew all 10 of its matches against the example player and lost all 10 to CEIA.

4 Analysis and Discussion

Reward shaping was the main reason PPO started learning at all. With sparse reward, the agent almost never sees a goal event, so most policy-gradient updates carry little useful

	Ex. player	CEIA	Ag. 1 (rew. shaped)	Ag. 2 (self-play)
Agent 1: reward-shaped	10/0/0	8/0/2	—	10/0/0
Agent 2: self-play	0/0/10	0/10/0	—	—
Agent 3: LUCKETS (ours)	10/0/0	8/2/0	7/1/2	10/0/0

Table 1: Head-to-head record (wins / losses / draws), 10 matches per pairing. Each row is the blue team; column is the orange opponent. "—" = symmetric duplicate not run.

Figure 2: LUCKETS_AGENT (blue) vs. CEIA baseline (orange) during evaluation; score 5–1 mid-game.

information. The delta rewards change this: the agent gets a small, informative signal whenever it moves closer to the ball or pushes the ball forward. Using deltas instead of absolute distance was important. An absolute-distance reward would penalise the agent for not standing on top of the ball, which we found in early experiments produces ball-chasing rather than positioning. The clearest evidence is the contrast between Phases 1 and 2: identical algorithm, similar step budget, but only the shaped agent learned to score.

The curriculum improved the final policy, but only after we fixed the opponent-mapping design. The clearest evidence is the head-to-head record against Agent 1: LUCKETS wins 7, loses 1, draws 2, despite both agents using the same shaping. We think this helps because of distribution shift. A policy trained only against random opponents converges to a narrow set of states; the CEIA baseline's ball-approach patterns and defensive positions are partly off that distribution. Stages 1–2 move training toward those patterns, and Stage 3 keeps the distribution expanding as the policy improves. Figure 1d shows the staircase we expected: win rate drops at every stage advance and then recovers.

The first curriculum failed for a practical implementation reason, not a methodological one. The win rate looked strong, but the agent never actually left Stage 0. The promotion rule fired correctly; the problem was that the opponent policy used by the Ray workers did not change. With a single shared `opponent` policy, weight swaps at runtime require copying tensors into every worker's local config, and our callback only mutated the driver's copy. This was misleading at first because the training metrics looked exactly like a successful curriculum. The fix in Phase 4 is structural: by declaring all three opponent policies up front, the swap is just a routing change in `policy_mapping_fn` rather than mid-training tensor mutation.

Pure self-play failed because there was no gradient to bootstrap from. Without shaping, neither side generates a scoring trajectory early enough to provide a learning signal. Both teams converge on idle behaviour with zero goals, entropy collapses, and rotating frozen snapshots cannot perturb the fixed point because the reward signal for escaping it is exactly zero. Reward shaping provides the bootstrap. The curriculum gives that bootstrap a useful target to grow toward.

Limitations. Both shaped agents stop at 8/10 against CEIA, not 9/10. Agent 1 turns its two losses into draws instead, which suggests LUCKETS is more aggressive in a way that wins more head-to-heads but occasionally costs a goal against a strong defender. Shaping also ignores teammate position, so there is no passing incentive. A symmetric promotion/demotion rule (advance at ≥ 0.8, retreat at ≤ 0.4) is the most natural next step.

Acknowledgments

We thank the course staff for the SoccerTwos starter kit [5] and the CEIA baseline, and the Georgia Tech PACE team for cluster access.

References

[1] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In International Conference on Machine Learning (ICML), 2009.

[2] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica. RLlib: Abstractions for distributed reinforcement learning. In International Conference on Machine Learning (ICML), 2018.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

[4] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In International Conference on Machine Learning (ICML), 1999.

[5] B. Oliveira. Soccer-twos: a starter kit for Unity ML-Agents SoccerTwos. https://github.com/bryanoliveira/soccer-twos-env, 2021.

A Hyperparameters

Hyperparameter	Value
Algorithm	PPO (Ray RLlib 1.4.0, PyTorch)
Policy/value network	MLP $[256, 256]$, ReLU, shared trunk + value head
Learning rate	3×10^{-4}
Discount γ	0.995
GAE λ	0.95
PPO clip ϵ	0.2
Entropy coefficient	0.01
Value-loss coefficient	0.5
SGD minibatch / epochs	512 / 10
Train batch size	12,000
Rollout fragment	1,000
Workers $\times$ envs/wkr	$12 \times 3 = 36$
Total env steps	$\sim$10M (Agent 1); $\sim$2.4M (Agent 2); $\sim$35.8M (Agent 3)
Shaping α_{prox} (Agents 1, 3)	0.02
Shaping α_{prog} (Agents 1, 3)	0.05
Promotion gate (Agent 3)	win rate ≥ 0.80 for 3 consecutive iters

Table 2: Final-run hyperparameters. Shaping coefficients are zero for Agent 2.